

12 OCTOBER 2025

POT JS

MANUAL

POT JS is a library to effortlessly store, update and retrieve values to a blockchain-based storage.

Under the hood it is a Javascript API for the Go Proximity Order Tree (POT) implementation that allows to write and read key-value pairs to and from the Swarm decentralized storage network.

A POT storage tree structure allows to have updateable key-value maps stored on an otherwise immutable, decentralized, content-addressable network like Swarm. It is called Proximity Order Tree after a fundamental, new storage technique that is described in a forthcoming paper.

This manual covers the use of POT JS as well as details for maintainers and contributors to the project.





INDEX

USE	3
QUICK START	3
<i>Example</i>	3
INTRODUCTION	4
<i>Brief</i>	4
<i>Proximity Order Tree</i>	5
<i>Coding</i>	6
<i>Error Propagation</i>	13
<i>Crashes</i>	13
<i>Further Reading</i>	14
TUTORIAL	15
<i>Node App</i>	21
FUNCTIONS	33
EXAMPLES	35
<i>Example 1: Minimal</i>	35
<i>Example 2: Interaction</i>	36
<i>Example 3: Initialization</i>	39
<i>Example 4: Synchronous</i>	40
<i>Example 5: Network</i>	42
<i>Example 6: Network II</i>	43
<i>Example 7: Node</i>	44
<i>Examples 8 and 9: Web App</i>	46
<i>What You Should See</i>	48
BUILDING	50
<i>What You Should See</i>	50
CLEAN BUILD	51
REQUIREMENTS	52
<i>POT JS API</i>	52
<i>GO POT</i>	52
TESTS	53
<i>What You Should See</i>	53
DEVELOPING	57
FILES	58
MAKE	59
DEVELOPER NOTES	60
<i>A JS API implemented in Go</i>	60
<i>Notes on Debugging</i>	63
<i>Note on syscall/js</i>	64
<i>Initialization</i>	64
<i>Types</i>	65
<i>Tests</i>	67
<i>Error Handling</i>	67
<i>Slots</i>	68
<i>Node.js Swarm Persister</i>	72
<i>Synchronous Architecture</i>	73
ROADMAP	74

USE

QUICK START

- Drop `pot.wasm`, and `pot-web.js` or `pot-node.js` in a directory.
- Write some html5 or JS using functions `new()`, `put()`, `get()`, `save()`, and `load()`.
- Check out the function reference (separate doc) and examples.

There is no need to *build* the project, i.e., no need to install or run Go, to *use* it.

EXAMPLE

```
<script src="pot-web.js"></script>

<script>

(async () => {

    await pot.ready()
    map = await pot.new()
    await map.put("hello", "POT!")
    value = await map.get("hello")
    console.log("key <hello> has:", value)
})();

</script>
```

Check out the function overview and function reference (separate doc).



INTRODUCTION

This is an introduction to how POT JS is used. The basics are simple and fit on this page.

You may then chose to work through the structured explanation that follows to get a deeper understanding; or skip to the hands-on tutorial. You might decide to check out the (separate) function reference after a cursory look at this introduction, to beat your own path. The final chapters of this manual explain the inner workings of POT JS to contributors.

BRIEF

POT JS provides a simple way to store and retrieve mutable key-value pairs to and from the immutable, decentralized storage network Swarm.

The main primitives are:

```
kvs = await pot.new(swarm, batch)
await kvs.put("some key", "some value")
value = await kvs.get("some key")
```

This almost constitutes a complete example program, cf. example 1 (pg. 35).

The implementation caches the data in-memory. To write the entire key value store to a network, and retrieve data from it:

```
ref = await kvs.save()
kvs = await pot.load(ref)
```

That's pretty much it. There are a lot more function for raw byte access, synchronous access, and tests. To use POT JS include, for in-browser use:

```
<script src="pot-web.js"></script>
```

For node.js, require:

```
require("pot-node")
```



PROXIMITY ORDER TREE

POT JS allows to use Proximity Order Trie (POT) functionality in the browser or with node.js.

A proximity order tree is a data structure that allows to operate a key-value store on an immutable, content-addressed¹ storage network. It makes it possible to effortlessly use Swarm as a key-value database; or to store a map-like structure to it – and be able to change values.

This bags two oxymorons: an immutable storage would seem to not allow for any update of values nor adding of additional key-value pairs. And Swarm is content-addressed: a hash of the content (not an arbitrary key) constitutes the address of the content in the network, to retrieve it by.

POT solves this by exploiting a feature observed in Merkle tries when hashing its nodes: after an update, there is only one path back to the root that has to be re-hashed to arrive at a new root hash. All other hashes in the trie remain the same, as no values that are part of their pre-images have changed. But the root hash always changes. That's the point with Merkle trees. POT uses this attribute to implement an update to data stored in the proximity order trie as a change of the root hash.²

Because a network like Swarm functionally provides the inverse to the hashing process – it returns the pre-image to the hash, by the very definition of content-addressing – the update process of a Merkle trie can be repurposed as the re-indexing mechanism of a trie that stores content-addressed data. A new root hash will be created on every update that serves as the new entry point to the respectively now-valid tree of data entries. Deleted nodes are not present anymore, because they fell out of the tree, even if they are still stored. Updated values are sorted into the tree with their new address. All entries, old and new, still exist in-storage. But they are not part of the hash tree anymore. The nodes that held the key-value pair of an updated pair that now has a new value are left out as their combined key and value contents has changed its hash.

The keys of this key value storage, though, are not native Swarm keys, which is made for retrieving pre-images of hashes. To search for a specific key to retrieve its value is therefore not something Swarm's base layer can help with. POT uses specific bit patterns to implement navigation of this data store at a useful pace. That is, POT is the complex algorithm that makes the system 'understand' where to find the value to a key, as well as to understand where to insert a new value associated with a new key. Concretely, a proximity order trie uses the bit patterns of a key's hash to establish a logarithmic-time search order. It is described in a forthcoming paper.

POT JS uses the implementation of POT in Go that performs the actual number crunching for this:

<https://github.com/ethersphere/proximity-order-trie>

¹ Content-addressed means that the storage address is produced from the value that is stored. If the value changes, the address changes, too. A value can therefore not be updated without losing its address.

² The values of this KVS are not the values stored in Swarm. They have keys that are independent of their values. Key and value are stored together and constitute the value of the Swarm record.



CODING

INITIALIZATION

BROWSER

For use in the browser, to initialize, include **pot-web.js**, then wait for the initialization to finish:

```
<script src="pot-web.js"></script>
<script>
  (async () => {
    await pot.ready()
    ...
  })()
```

If a path to **pot.wasm** must be provided, use the **wasm** attribute:

```
<script src="pot-web.js" wasm="lib/pot.wasm"></script>
```

NODE.JS

For use with node.js, include **pot-node.js**, then wait for the initialization to finish:

```
require("pot-node")
(async () => {
  await pot.ready()
  ...
})()
```

If a path to **pot.wasm** must be provided, add a parameter to **require** like this:

```
require("pot-node")("lib/pot.wasm")
```



DATA STORAGE

After `pot.ready()` resolves, the JS `pot` object can be used to create a new key value store (KVS) with `pot.new()`. Use `put()` and `get()` functions on it to store and retrieve key-values pairs to it.

Because in this case the `new()` call has no arguments, **in-memory storage** is used for the KVS, which can be useful for testing and prototyping. In this mode, values are not permanently stored to a network but flushed and lost whenever the program is stopped.

```
map = await pot.new()
await pot.put("hello", "Hello, P.O.T.!")
value = await pot.get("hello")
```

See `examples/example1.html` for a complete program using these POT JS calls.

PERSISTING

To store the entire KVS, use `save()`, to retrieve it at a later point, `load()`.

```
ref = await kvs.save()
await pot.load(ref)
```

This only makes sense when using a network as storage. The in-memory mode will lose all storage when a program stops.

THE SAVE REFERENCE

The reference returned from `save()` identifies one specific state. It is not tied to the identity of a specific KVS and not tracking its changes. It will at all times return the same KVS state through `load()`, i.e., the same set of key-value pairs, until the postage batch expires and the data is purged from Swarm; or until a test network is shut down.

A different reference will be returned upon saving the same KVS after updating it (adding or changing values).

The same reference will retrieve the same KVS, including when the KVS that was originally saved is later updated and saved again or not, or a KVS that was loaded through this reference is updated, and saved again or not. Any of these updated KVSs would have a different reference returned from `save()` after any update.



Creating the exact same KVS by the same sequence of `put()` calls will result in the same reference when saving.

All this is true when using in-memory storage the same as when storing to a network.

NETWORK CONNECTION

To connect with a Swarm network, the call to `pot.new()` requires:

1. a URL to a Swarm bee node http API and
2. a Swarm batch ID to pay for storage.³

For example (cf. `example5.html`), the following are literals that connect with a local Swarm test network (more on this below):

```
map = await pot.new("http://localhost:1633",
    "ab642e87327ec31816ed7aa675b4e92ea2c3d92c5022a599604c8bd8ddfe545d")
```

Note that the batch id – the second argument – has to be freshly created for your tests.

Another example, for node.js (cf. `example8.js`):

```
bee = process.argv[2]
batch = process.argv[3]
kvs = await pot.new(bee, batch)
```

An example invocation of these lines would be:

```
% node hello.js http://localhost:1633 \
    6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa
```

Again, note that the batch id – the third argument – has to be freshly created for your tests.

BATCH ID

All tests and most examples can be tried out **without** a batch ID: POT has an in-memory mode that works without that. But for production use you will likely need one.

³ For an intro to Swarm and the concept of its network nodes, the Bees, and the postage batch IDs that power its incentive mechanisms, see <https://ethswarm.org>. You do not need to operate a Swarm Bee node to use POT JS. You can try it locally, with in-memory storage to develop. It can be operated with a local Swarm test network. But to write data to the decentralized Swarm mainnet, you will need a postage batch.



A batch ID stands for **postage stamps** that Swarm uses for incentivization, inter alia, to compensate its Bee nodes to keep copies of the stored data. Batch IDs are the prime payment mechanism that Swarm offers, pervading most every aspect of decentralized data hosting on Swarm.

It is **not** required to understand the concept of batch IDs for trying out POT JS, nor even for coding POT JS apps on a Swarm testnet: **make** rules set the testnet up and obtain a free batch id for your tests right after.

But if you are planning to work with Swarm, it will be indispensable to understand the concept of batch ids, how to buy them and for how much data and for how long they are good:

<https://docs.ethswarm.org/docs/develop/tools-and-features/buy-a-stamp-batch>

Batch IDs are required when working with a network instead of the in-memory mode. To test batch IDs, a local Swarm network can be installed.

INSTALLING A LOCAL SWARM NETWORK

All tests and most examples can be tried out **without** connection to any Swarm network: POT's in-memory mode allows for testing and developing without one.

But to test an app with a Swarm network, the easiest way is to install a local Swarm test network. The recommended way is to use the docker-based installation by fairdatasociety:

<https://www.npmjs.com/package/@fairdatasociety/fdp-play>

Alternatively, Swarm provides documentation on how to set up a local 2-node testnet:

<https://docs.ethswarm.org/docs/develop/tools-and-features/starting-a-test-network/>

Once the network is up, the following script creates a new batch ID and stores it into the file `.batch_id`.⁴ However, using `make` to initiate tests or examples does that for you.

```
# starting five local swarm nodes
fdp-play start --detach

# buy stamps (free in this test setup)
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
echo "postage batch ID: $(cat .batch_id)"
```

⁴ The file name starts with a dot and thus, be default, is not visible in Mac Finder, nor with `ls` in the terminal. Press **shift + command + .** or use `ls -a`.



```
# stopping nodes
fdp-play stop
```

The Makefile rule for example 9 (search for “**example9:**” in the file **Makefile**) is an example for how to start a local Swarm test network and create a postage batch for it on it. This batch is a free batch of stamps, created on your own local Swarm test network.

EXPLICIT INITIALIZATION

As an alternate way to initialize WASM for in-browser scripts, you may leave out **potjs.js**: include only the generic Go WASM wrapper `wasm_exec.js`, create a `Go` object and instantiate the POT WASM object **pot.wasm**, then run it once it is loaded. See **examples/example3.html**:

```
const go = new Go()
WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
  .then((r)=>{
    go.run(r.instance)
    map = pot.newSync()
  })
...
```

Then use **onWasmLoaded()** for the start of your POT JS use:

```
async function onWasmLoaded() {
  map = await pot.new()
  await map.put("hello", "POT!")
  value = await map.get("hello")
}
```



VERBOSITY

The default log level is **DEBUG**, printing details of state change to browser console or screen.

0	pot.NONE	no logging
1	pot.CRITICAL	logs only errors that would appear to arise from a malfunction of POT JS.
2	pot.ERROR	programming and runtime errors are also logged.
3	pot.INFO	general runtime information is logged.
4	pot.DEBUG	specific data, like put and get keys and values are logged.
5	pot.TRACE	Some code is wired to report on every step of the way.

The verbosity can be adjusted with **pot.setVerbosity()**. The log prints to screen when running POT JS with node.js. In the browser, it logs into the browser console.

Cf. examples/example8.js:

```
(async () => {  
    await pot.ready()  
    pot.setVerbosity(pot.INFO)  
    kvs = await pot.new()  
    ...  
})();
```

There are three ways to set verbosity before the WASM executable is even initialized, to suppress its very first output: setting **globalThis.potjs_verbosity**; adding the attribute **verbosity** to the script tag of **pot-web.js** in the browser; or setting a second parameter to **require** of **pot-node.js** in node.js. These latter ways to set verbosity all need an integer supplied, the **pot.*** constants listed above do not exist yet at that point.

Cf. examples/examples7.js:

```
var go = new Go()  
global.potjs_verbosity = 3  
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)  
    .then((r) => { go.run(r.instance) })  
  
onWasmLoaded = async () => {  
    ...  
}
```



Cf. `examples/examples7.js`:

```
var go = new Go()
global.potjs_verbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"),
go.importObject)
  .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {
  ...
}
```

Cf. `examples/examples7.js`:

```
var go = new Go()
global.potjs_verbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"),
go.importObject)
  .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {
  ...
}
```

Note that `potjs_verbosity` can only be used before the WASM executable is initialized and will not have any effect after. It is meant for the case that neither `pot-web.js` not `pot-node.js` are used.

Cf. `examples/example7.js`:

```
var go = new Go()
global.potjs_verbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"),
  go.importObject)
  .then((r) => { go.run(r.instance) })
onWasmLoaded = async () => {
  ...
}
```



ERROR PROPAGATION

The error strategy of the API is JS-Style, using JS exceptions triggered from the go wasm functions via promise mechanisms, rather than returning error codes, Go-style.

The implementation of this is in Go though (`potjs.go`), not in JS, creating JS code in Go, with full data/situational access on the Go side. A major challenge is that Javascript exceptions cannot be thrown from Go. What works is to call the reject parameter of in the executor of JS promises that are constructed in Go. Because the most relevant functions are implemented like this at any rate, this covers most cases.

Synchronous functions, however, in part return JS error objects that have to be tested for where an exception would normally be expected.

CRASHES

The Go WASM executable, `pot.wasm`, can deadlock and crash, in which case it cannot be recovered. All POT JS functions stop working and throw an error instead that reports that the Go executable is down. In this case, `pot.wasm` must be re-initialized before the POT JS functions can be used again. This affects all `pot` and `map` functions, it means that while everything else might still work, literally the methods of a `map` that on the face look like native Javascript functions, are dead.

Reasons for crashing are deadlocks and uncaught panics on the side of the Go executable. Multiple measures are in place to keep the executable alive in all scenarios, to handle panics gracefully and to not allow a deadlock to happen. To maximize robustness, one might still want to program aware of this case, which means to avoid synchronous functions in production.

The capability to deadlock is not a fault of the Go POT implementation but presumably lies in the single-thread nature of Javascript that can be programmed highly asynchronously but is not truly parallel, multi-threaded, or multi-process (except when using the duly limited Web Workers). That Javascript is designed this way provides strong safeguards against a class of difficult to track errors. But they are coming off when crossing from Javascript into Go-land *and back*: a blocking wait in Go that hands control back to the browser deadlocks its main Javascript execution thread. Javascript waits for Go to continue and Go for Javascript. This is reliably the case when Go executes a network fetch that is handed back from WASM to Javascript to execute. When using POT JS to write to and



read from Swarm, almost every function contains a fetch. When programming against one of the in-memory persistence models, none does.

FURTHER READING

Refer to the examples in the `examples/` folder and or the Examples chapter to understand how POT JS might be used in the specific use case you envision.

The function reference in the `doc/` folder explains all available functions with additional example snippets.

The tutorial visits a number of relevant angles to get a fuller picture.



TUTORIAL

This is a step-by-step guide on how to use POT JS in the browser or with node.js.

We'll discuss the building blocks, alternative options, and aspects that deserve attention.

We will start with a minimal in-browser example, develop it into a more interactive experience that is the prototype of a web app, shifting functionality from the browser to the server-side to show both options.

If you are really fast, checking out `examples/example1.html` may be all you need to get going; then use the **function reference** document in `doc/`.

INSTALLATION

Create a new directory, and clone POT JS into it from github:

```
% mkdir tut ; cd tut
% git clone https://github.com/brainiac-five/potjs
```

There is no building or installation required beyond this.

The files we need are `lib/pot.wasm`, `lib/pot-web.js` and `lib/pot-node.js`. These are, the Go implementation with JS interface compiled to WASM, and the Javascript initialization code for it, which differs between in-browser and node.js. The files are portable across operating systems and browsers.

Create a file `t1.js` and paste or type this:

```
require("./potjs/lib/pot-node")

; (async () => {
  await pot.ready()
  kvs = await pot.new()
  await kvs.put("hello", "node")
  value = await kvs.get("hello")
  console.log("hello:", value)
})
```

The fourth line blocks until the initialization of POT JS is complete.



```
await pot.ready()
```

The initialization is kicked off by the **require**. It loads the WASM executable **pot.wasm** that has the number crunching that organizes the key-values. The file is started and does not stop during the lifetime of **t1.js** but waits for calls to it.

The loading and starting of **pot.wasm** is an asynchronous affair though. And before the relevant pot calls can happen, the initialization needs to be complete. That's what **pot.ready()** waits for.

```
kvs = await pot.new()
await kvs.put("hello", "node")
value = await kvs.get("hello")
console.log("hello:", value)
}
```

```
kvs = await pot.new()
await kvs.put("hello", "node")
value = await kvs.get("hello")
console.log("hello:", value)
}
```

```
kvs = await pot.new()
await kvs.put("hello", "node")
value = await kvs.get("hello")
console.log("hello:", value)
}
```

```
kvs = await pot.new()
await kvs.put("hello", "node")
value = await kvs.get("hello")
console.log("hello:", value)
}
```




WHAT YOU SHOULD SEE

```
pot: » POTWASM
pot: » node.js detected
pot: » init done
pot: » ready
pot: » initialize new P.O.T.
pot: » created new in-memory persister
pot: » using in-memory persister
pot: > slot ref: 1
pot: » put hello: node
pot: > → 68656c6c6f00000000000000000000000000000000000000000000000000000000: 036e6f6465
pot: » get hello: node
pot: > ← 68656c6c6f00000000000000000000000000000000000000000000000000000000: 036e6f6465
hello: node
```

STANDARD OPERATIONS

The main functions of POT JS are `put()` and `get()`. Maps are created with `new()`. They are saved to the network (or to in-memory POT if that is elected) using `save()` and retrieved with `newByReference()`.

We add a write and a read to our example. In `onWasmLoaded()` under `READY`. add:

```
map = await pot.new()

await map.put("hello", "POT!")

value = await map.get("hello")

console.log("key <hello> has:", value)
```

The first line creates a new key-value-store. What is special about POT is that it can be stored on an immutable storage network like Swarm. For our test though, we are using an in-memory storage for now.

The next line puts the string “POT” in the store, retrievable under the key “hello”.

The next line does exactly that: retrieve the value of key “hello”, which should be “POT”.

The last line prints the resulting **value** on the browser console, which will be “POT”.

To try it out, serve the directory, e.g., ...:

```
% npx http-server -c1 .
```



... and open the page:

http://localhost:8080/t1.html

The page will again be blank.

If you open the developer console, you should see:

```
pot: » POTWASM  
pot: » init done  
t1.html:12 READY.  
pot: » initialize new P.O.T.  
pot: > created new in-memory persister  
pot: > using in-memory persister  
pot: slot ref: 1  
pot: » ready  
pot: » put hello: POT!  
pot: » → 68656c6c6f00000000000000000000000000000000000000000000000000000000: 03504f5421  
pot: » get hello: POT!  
pot: « ← 68656c6c6f00000000000000000000000000000000000000000000000000000000: 03504f5421  
t1.html:20 key <hello> has: POT!
```

browser console of tutorial/t1-step-2.html

The middle part of the log after **READY.** is all about a new key-value-store being created. It is received in the variable **map** on the Javascript side.

The line `put hello: POT!` echoes the arguments that `kvs.put()` received.

Below, the digital values that are actually stored are shown.

`68656c6c6f0000000000000000000000000000000000000000000000000000` stands for the padded key “hello” and `03504f5421` for “POT!”. Keys are encoded differently than values. If the value to be stored was “hello”, instead of “POT!”, it would be `0368656c6c6f` instead of `03504f5421`.

The get log entry is showing the key AND the value the `get()` finds for it, not just its key parameter.

On the console, you should again see the logs of the three assets the http server provided:

```
[2025-09-13T12:21:26.674Z] "GET /t1.html" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:6487) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-09-13T12:21:26.732Z] "GET /potjs/lib/wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-09-13T12:21:26.819Z] "GET /potjs/lib/pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```



part of the terminal output of tutorial/t1-step-2.html

→ you stored and retrieved a value from a POT.

PREMADE INITIALIZATION

You can include `potjs.js` to skip over having to type out the standard initialization. The easiest way to explain what that does is to show `potjs/lib/potjs.js`:

```
const go = new Go()

if(typeof pot == 'undefined') pot = {}

pot.start = new Promise((resolve, reject) => {
  let wasm = document.currentScript.getAttribute('wasm')
  if(!wasm) wasm = "pot.wasm"
  WebAssembly.instantiateStreaming(fetch(wasm), go.importObject)
    .then((r) => { go.run(r.instance) ; resolve(pot) })
    .catch((e) => { reject(e) })
})

pot.ready = () => { return pot.start }
```

lib/potjs.js

It wraps the initialization steps we discussed above into a promise and allows to inquire via `pot.ready()` whether the initialization has **started**. This is not quite as precise as waiting for the call of `onWasmLoaded()` and will be improved going forward because it can be too early.

However, using `potjs.js` our tutorial code can be abbreviated to:

```
<script src="potjs/lib/wasm_exec.js"></script>
<script src="potjs/lib/potjs.js" wasm="potjs/lib/pot.wasm"></script>

<script>

  async function onWasmLoaded() {

    console.log("READY.")

    map = await pot.new()
```



```
    await map.put("hello", "POT!")

    value = await map.get("hello")

    console.log("key <hello> has:", value)
  }

</script>
```

tutorial/t1-step-3.html

This code works exactly like the version before, giving the same logs.

SUB-COMPONENT INTEGRITY

To safeguard the integrity of the included scripts – but not the WASM script – fingerprints can be hardcoded into the HTML.

The required SHA-384 hashes can be found in `potjs/lib/*.sha384`. A “sha384-” must be prepended for the integrity tag.

The first two lines become:

```
<script src="potjs/lib/wasm_exec.js" integrity="sha384-
PWCs+V4BDf9yY1yjkD/p+9xNEs4iEbuVq+HezA0JiY3XL5GI6VyJXMsvnjwNbce"></script>
<script src="potjs/lib/potjs.js" wasm="potjs/lib/pot.wasm" integrity="sha384-
woy0Fou09EcyFemKbNcnl+HI7XRxaENJQgJPdgPuBMqaqXwTWzUgF+zLViGzeQQF"></script>
```

beginning of tutorial/t1-step-4.html

This code works exactly like before, showing the same results.

ERROR CONDITIONS

The WASM executable is not a library but a Go program that runs continually, waiting for the next request. If it crashes, the functionality it extends just goes away. The main methods of JS objects that POT JS defines become unavailable.

It is relatively easy to deadlock the Go WASM by using the synchronous functions in the wrong context. Those are the POT JS functions ending on **SynC**. They are generally not safe to use when using a Swarm network as storage backend, that is, in the standard case.



In case the Go WASM program goes down, it has to be re-initialized.

NODE APP

For the next step, we will look at how POT JS works with node.js instead of the browser.

We'll create a single-script web app next that can persist data it handles server-side on Swarm. The result is going to be almost identical to example 8 (`examples/example8.js`).

Create a new folder, change into it and clone potjs into it.

```
% mkdir t2 ; cd t2
% git clone https://github.com/brainiac-five/potjs
```

INSTALLATION

For clarity, copy the two executables we need out of the repo and delete the rest.

```
% cp potjs/lib/pot.wasm .
% cp potjs/lib/wasm_exec.js .
% rm -rf potjs
```

Create a node.js/potjs test script `t2.js` and run it.

```
fs = require("fs");
require("./wasm_exec");

var go = new Go();

WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)
  .then((r) => { go.run(r.instance) })

onWasmLoaded = async () => {

  map = await global.pot.new()
  await map.put("hello", "node")
  value = await map.get("hello")
  console.log("hello:", value)
}
```



```
./tutorial/t2-step-1.js
```

The filesystem **fs** is required for **readFileSync()**, to read the wasm file.

The generic **wasm_exec** is also the same as for use in the browser.

As before, a `Go` object defined in `wasm_exec.js` is created; and when loading is done, the WASM executable is `run`.

Our program starts when `onWasmLoaded()` is called from `pot.wasm` as soon as it is done with initializations. A new `map` is created, to which a key and value are `put()`, then `get()` reads it back out.

Finally, the key, and value, as retrieved, are printed to the console.

Running it

```
% node t2.js
```

your should see:

[illegible]

The `pot.wasm` library is started and logs the two first lines.

Then a new map is created, to which a key and value are written, then read back out.

The log shows the binary values that are written and read to the POT.

Eventually, the key and value are logged to the console.

Our starting point is thus basically `example/example7.js`.



HTTP SERVER

We add a minimal web server.

To the t2.js script append:

```
const http = require('http')
const page = "Hello, Swarm!"
const server = http.createServer(function(request, response) {

  response.writeHead(200, {'Content-Type': 'text/html'})
  response.end(page)
})
.listen(3000, '127.0.0.1')
```

part of ./tutorial/t2-step-2.js

Run it

```
% node t2.js
```

Open a browser and point to **http://localhost:3000**.

```
% open http://localhost:3000
```

It should show

```
Hello, Swarm!
```

INPUT FORM

Stop the server and enhance the definition of the **page** variable to become an input form:

```
const page = `
<pre>

  <form method=post action="http://localhost:3000">

    key    <input name=key>

    value  <input name=value>
```



```
        <input type=submit name=button value=put>

<hr />

    key    <input name=search>

        <input type=submit name=button value=get>

    value  <input name=result>

    </form>

</pre>
```

part of ./tutorial/t2-step-3.js

Running the server again, reloading the page <http://localhost:3000> in the browser you should see:

POT JS Example 8: Node.js Web App

key	
value	
	put

key	
	get
value	

screen of ./tutorial/t2-step-3.js

INTERACTION

To interact with the form in the browser, we'll first clean up marginally. Change the WASM hook to:



```
global.onWasmLoaded = () => {  
  
  kvs = pot.newSync()  
}
```

part of ./tutorial/t2-step-4.js

Replace the request handler, **server()**, with:

```
const http = require('http')  
const qs = require('querystring')  
  
const server = http.createServer(function(request, response) {  
  
  if (request.method == 'POST') {  
  
    var body = ''  
    request.on('data', function(data) {  
      body += data  
    })  
  
    request.on('end', async function() {  
  
      var value = "put"  
      post = qs.parse(body)  
  
      if(post.button == 'put')  
        await kvs.put(post.key, post.value)  
      else  
        value = await kvs.get(post.search)  
  
      response.writeHead(200, {'Content-Type': 'text/html'})  
      response.end(value)  
    })  
  }  
  else {  
    response.writeHead(200, {'Content-Type': 'text/html'})  
    response.end(form)  
  }  
})  
.listen(3000, '127.0.0.1')
```



The following line reads the value entered in the second form field and writes it to the POT. The first form field is used to enter the key that it is saved under.

```
await kvs.put(post.key, post.value)
```

In the browser, this looks like this: e.g., entering **A** for the key, and **123** for the value.

key
value

After clicking put, the browser then simply shows one word, 'put'. Meanwhile, `kvs.put()` is triggered in the server and the value `123` is stored under key `A`.

Where it is actually stored is the **pot.wasm** executable that keeps running all the time while the server operates. It is stored as in-memory key-value-store for now, which is a good thing for testing.

What you should see now in the terminal is this:

[illegible]

To get back to the form, use the browser's back button.

If you enter **A** in the third form field, and click get, the browser will show `'123.'`

The terminal will add:

```
pot:  » get A: 123
```




```
pot: » init done
```

To also eliminate the remaining first two lines, a different method has to be used. The global variable `potjs_verbosity` needs to be set before WASM is initialized, e.g., like this:

```
var go = new Go()
globalThis.potjs_verbosity = 0
WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })

global.onWasmLoaded = () => {
    kvs = pot.newSync()
}
```

part of `./tutorial/t2-step-6.js`

This suppresses even the first two lines coming from POT JS.

The value set to `globalThis.potjs_verbosity` must be a number, a literal or a variable you defined but `pot.NONE` etc. do not exist yet at this point. The initialization of WASM creates them.

The value in `globalThis.potjs_verbosity` matters at the point when the `pot.wasm` is initialized and changing it has no effect after that point. The function `pot.setVerbosity()` can be changed without immediate effect, any time.

SINGLE-PAGE

We'll add a little more polish by staying on the page.

After the `onWasmLoaded()` function, insert two functions, `put()` and `get()`:

```
async function put(key, value) {

    await kvs.put(key, value)

    return `put ${key}: ${value}`
}
```



```
async function get(search) {  
  
  value = await kvs.get(search)  
  
  return `get ${search}: ${value}  
    <script>  
      document.getElementsByName('result')[0].value = '${value}'  
    </script>`  
}
```

part of ./tutorial/t2-step-5.js

Then replace the request hook with calls to the **put()** and **post()** functions we just defined.

They are replacing the direct calls to **kvs.put()** and **kvs.get()**.

We are also adding a variable, **log**, to gather a minimal status report.

```
request.on('end', async function() {  
  
  var log  
  const post = qs.parse(body)  
  
  if(post.button == 'put')  
    log = await put(post.key, post.value)  
  else  
    log = await get(post.search)  
  
  console.log("srv: " + log.match(/.*/)[0])  
  
  response.writeHead(200, {'Content-Type': 'text/html'})  
  response.end(form + '<hr><br><pre>\t\t' + log)  
})
```

part of ./tutorial/t2-step-5.js

The complete code we arrived at is:

```
const page = `<pre>  
  
  <form method=post action="http://localhost:3000">
```



```
    key    <input name=key>

    value  <input name=value>

           <input type=submit name=button value=put>

<hr />

    key    <input name=search>

           <input type=submit name=button value=get>

    value  <input name=result>

</form>

</pre>`

const fs = require("fs")
require("./wasm_exec") // note the ./

var go = new Go()
var kvs

WebAssembly.instantiate(fs.readFileSync("./pot.wasm"), go.importObject)
  .then((r) => { go.run(r.instance) })

global.onWasmLoaded = () => {

  kvs = pot.newSync()
}

async function put(key, value) {

  await kvs.put(key, value)

  return `put ${key}: ${value}`
}

async function get(search) {

  value = await kvs.get(search)

  return `get ${search}: ${value}`
}
```



```
<script>
  document.getElementsByName('result')[0].value = '${value}'
</script>`
}

const http = require('http')
const qs = require('querystring')

const server = http.createServer(function(request, response) {

  if (request.method == 'POST') {

    var body = ''
    request.on('data', function(data) {
      body += data
    })

    request.on('end', async function() {

      var log
      const post = qs.parse(body)

      if(post.button == 'put')
        log = await put(post.key, post.value)
      else
        log = await get(post.search)

      console.log("srv: " + log.match(/.*\/)[0])

      response.writeHead(200, {'Content-Type': 'text/html'})
      response.end(page + '<hr><br><pre>\t\t' + log)
    })
  }
  else {
    response.writeHead(200, {'Content-Type': 'text/html'})
    response.end(page)
  }
})
.listen(3000, '127.0.0.1')
```

./tutorial/t2-step-5.js

Running it



```
% node t2.js
```

We see the form fields as before, but pressing buttons does make the page disappear.

The result of the get is shown in the fourth form field.

Both put and get results are shown in the bottom line.

This is what you will see, although not at the same time:

key

value

key

value

put B: 5

screens of ./tutorial/t2-step-5.js

CONCLUSION

POTJS works with node.js as easily as it works in the browser. It doesn't get in the way and is a simple way to store state of a web app to a trustless, decentralized network.



FUNCTIONS

The focus of the functions are `get()` and `put()`. They are split in two by two main groups, for four different ways of calling. Additionally, there are `init`, `test` and `support` functions.

The groups are asynchronous / synchronous and typed / raw respectively. Asynchronous functions return promises, synchronous functions block and return a value, error or null. Typed functions mark the Javascript type in the storage, to automatically return the right type on retrieval. This can be boolean, number,⁵ or string. Raw functions accept a Uint8Array of bytes to store and the right function has to be called to get the expected type back out. The synchronous functions' names end on `Sync`. The raw functions have `Raw` as a name component. The most relevant functions are `put()` and `get()`, which are asynchronous, typed functions. That means, `get()` returns a promise for the value to be retrieved and it will automatically be in the type that it was stored with `put()`.

Synchronous functions can be handy for prototyping and special situations but generally only work for tests in the browser using the in-memory persistence of the Go POT implementation, not when connecting to Swarm or a test network, and not when using node. It is possible that they might work well with the background threads of Web Workers but this has not been tested. Therefore, they should simply NOT be used in production. The asynchronous functions work in all circumstances.

Raw functions will be useful in special situations, whenever binary data is handled, but usually the vanilla (typed, asynchronous) functions `new`, `save`, `put` and `get` will be the first choice.

For details and examples, see file **Pot – (III) 31 August 2025 function reference.pdf**.

MAP INITIALIZATION

<code>pot.new</code>	create a new Swarm key-value-store, i.e., a map, async
<code>pot.newByReference</code>	create a new Swarm map from a reference of a prior save, async
<code>map.save</code>	store a Swarm KVS to the network, async
<code>pot.newSync</code>	create a new Swarm key-value-store, i.e., a map, sync
<code>pot.newByReferenceSync</code>	create a new Swarm map from a reference of a prior save, sync
<code>map.saveSync</code>	store a Swarm KVS to the network, sync

TYPE-AWARE

<code>put</code>	return a promise for null or Error. Asynchronous.
<code>get</code>	return a promise for the typed get, the type will be as put. Async.
<code>putSync</code>	return a promise for null or Error. Synchronous.
<code>getSync</code>	return a promise for the typed get, i.e., the type will be as put. Sync.

⁵ Javascript does not have native integers and stores all numbers as IEEE 754 64-bit floats internally.



RAW BYTE ACCESS

<code>putRaw</code>	put a raw <code>Uint8Bytes</code> array, async.
<code>getRaw</code>	get a raw <code>Uint8Bytes</code> array, async.
<code>getBoolean</code>	get the raw value as Boolean. First byte 0 for false, all else true, async.
<code>getNumber</code>	get the raw value as floating point number. Must be 8 bytes. Async.
<code>getString</code>	get the raw value as string, async.
<code>putRawSync</code>	put a raw <code>Uint8Bytes</code> array, synchronous.
<code>getRawSync</code>	get a raw <code>Uint8Bytes</code> array, synchronous.
<code>getBooleanSync</code>	get the raw value as Boolean. First byte 0 for false, all else true, sync.
<code>getNumberSync</code>	get the raw value as floating point number. Must be 8 bytes. Sync.
<code>getStringSync</code>	get the raw value as string, synchronous.

POT INITIALIZATION

<code>pot.ready</code>	promise that resolves when <code>pot.wasm</code> is initialized. Include <code>potjs.js</code> .
<code>onWasmLoaded</code>	called, if it exists, by <code>pot.wasm</code> when it is initialized. Without <code>potjs.js</code> .
<code>pot.setOptimization</code>	controls the extent to which POT JS writes to the log, a value from 0 to 5.
<code>globalThis.potjs_optimization</code>	setting optimization before initialization
<code>pot.setVerbosity</code>	controls the extent to which POT JS writes to the log, a value from 0 to 5.
<code>globalThis.potjs_verbosity</code>	setting verbosity before initialization

TESTING AND SUPPORT

These functions are not used in production. They serve to test the integrity of POT JS.

<code>log</code>	write to browser console via Go to verify a language-land roundtrip
<code>hello</code>	write hello to browser console to verify established WASM stream
<code>randKey</code>	32 random bytes for key testing
<code>randValue</code>	79 to 101 random bytes for value testing (range from Go POT)
<code>type_encoded_bytes</code>	JS type detection and value encoding with correct leading type byte
<code>type_decoded_value</code>	JS type detection of raw kvs value by leading type byte
<code>hangingPromise</code>	blocking promise (in Go) with time out for exception testing
<code>setFail</code>	trigger the next mock storage function called to return an error
<code>setPanic</code>	trigger the next mock storage function called to raise a panic
<code>setDelay</code>	trigger the next mock storage function called to stall for a given time
<code>setHang</code>	trigger the next mock storage function called to stall indefinitely



EXAMPLES

To run the minimal example, serve the main directory, e.g., with ``npx http-server .`` and open ``http://127.0.0.1:8080/example1.html``.

Or use

```
make example1
```

etc. There are nine examples 1-9 that can be started this way.

example1.html	minimal in-browser example as above, in-memory storage
example2.html	interactive in-browser example with input fields, integrity
example3.html	like example 2 without using pot-web.js
example4.html	like example 1 using synchronous access functions
example5.html	like example 1 but with local Swarm network storage
example6.html	like example 5, parametrized, with WASM path and log level 2
example7.js	like example 1 but for node.js
example8.js	similar to example 2, interactive web app for node.js, in-mem.
example9.js	identical to example 8, called to use a local Swarm network

EXAMPLE 1: MINIMAL

The **minimal** example, a simple put and get, logging to browser console.

- in-browser (code that is for node.js instead, uses **pot-node.js**)
- in-memory (to not be in-memory, **pot.new()** would need to receive arguments, see above.)
- asynchronous (asynchronous functions all have synchronous corollaries that end in **Sync()**.)

examples/example1.html

```
<script src="lib/pot-web.js"></script>
<script>
(async () => {
  await pot.ready()
  map = await pot.new()
  await map.put("hello", "POT!")
  value = await map.get("hello")
  console.log("key <hello> has:", value)
})();
</script>
```



WHAT YOU SHOULD SEE

BROWSER CONSOLE:

```
pot: » POTWASM
pot: » browser detected
pot: » init done
pot: » ready
pot: » initialize new P.O.T.
pot: > created new in-memory persister
pot: > using in-memory persister
pot: > slot ref: 1
pot: » put hello: POT!
pot: > → 68656c6c6f00000000000000000000000000000000000000000000000000000000: 03504f5421
pot: » get hello: POT!
pot: > ← 68656c6c6f00000000000000000000000000000000000000000000000000000000: 03504f5421
key <hello> has: POT!
```

EXAMPLE 2: INTERACTION

Interactive use, verbosity setting and integrity checks. A html form accepts input and shows results. The action is handled entirely in the browser. The sample KVS is stored in an in-memory storage.

The **script** tag in this example has three additional attributes. The first one, **wasm**, is the path to the **pot.wasm** file. It is unnecessary in this case as **pot.wasm** is looked for by default in the same folder as **pot-web.js**.

The example also shows how to set verbosity in the script tag, using the **verbosity** attribute. See the instructions, pg.11, for the list of valid arguments.

The **integrity** attribute accepts a hash of the included script **pot-web.js**.

- in-browser
- in-memory
- asynchronous

examples/example2.html

```
<pre>

POT JS Example 2: Interactive & Integrity

key   <input id=key>
value <input id=value>
      <button onclick="put()"> put </button>
```



```
<hr />

key    <input id=search>
      <button onclick="get()"> get </button>
value  <input id=result>

<img src=favicon.ico>

</pre>

<script src="lib/pot-web.js" wasm="lib/pot.wasm" verbosity=2
  integrity="sha384-j4t1zovTugIaitFBccUwbQLC7shu/4n2J0gsZZm66kCGYKM6Jo+A/B0mIZNx/5Zq">
</script>

<script>
var map
(async () => {
  await pot.ready()
  map = await pot.new()
})();

const field = (id) => { return document.getElementById(id) }

async function put() {
  key = field('key').value
  value = field('value').value
  await map.put(key, value)
}

async function get() {
  key = field('search').value
  value = await map.get(key)
  field('result').value = value
}
</script>
```

WHAT YOU SHOULD SEE

BROWSER

After 1) entering a value for *key* and *value*, and hitting *put*.



Then 2) entering the same key in the lower screen area and hitting *get* will yield the *value* in the lowest field.

POT JS Example 2: Interactive & Integrity

key

value

key

value



BROWSER CONSOLE:

```
pot: » POTWASM
pot: » browser detected
pot: » init done
pot: » ready
pot: » initialize new P.O.T.
pot: » created new in-memory persister
pot: » using in-memory persister
pot: » put a: b
pot: » get a: b
```



EXAMPLE 3: INITIALIZATION

More explicit initialization.

This page does not use `pot-web.js` but initializes `pot.wasm` directly.⁶

This example should have a mechanism that would prevent its `get()` and `put()` functions to be called before WASM initialization was complete. In practice buttons can't usually be pressed fast enough. In theory, `onWasmLoaded()` (see above) should be used for this check.

- in-browser
- in-memory
- asynchronous

`examples/example3.html`

```
<pre>

  POT JS Example 3: Explicit Initialization

  key    <input id=key>
  value  <input id=value>
         <button onclick="put()"> put </button>

<hr />

  key    <input id=search>
         <button onclick="get()"> get </button>
  value  <input id=result>

  <img src=favicon.ico>

<hr />

  This page does not use pot.js but initializes pot.wasm directly.

</pre>

<script src="lib/wasm_exec.js"></script>
```

⁶ The corollary is not possible for node.js with network storage.)



```
<script>

var map;
const go = new Go()
WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
  .then((r)=>{
    go.run(r.instance)
    map = pot.newSync()
  })

const field = (id) => { return document.getElementById(id) }

async function put() {
  key = field('key').value
  value = field('value').value
  await map.putSync(key, value)
}

async function get() {
  key = field('search').value
  value = await map.getSync(key)
  field('result').value = value
}

</script>
```

EXAMPLE 4: SYNCHRONOUS

This example page – that is otherwise like example 1 – uses synchronous calls that can be helpful for prototyping in the browser but don't work for network connections and not for use with node.

- in-browser
- in-memory
- synchronous

**examples/example4.html**

```
<pre>
```

POT JS Example 4: Synchronous

Check source and console.

This page uses synchronous calls that can be helpful for prototyping but don't work for network connections (only in-memory, i.e using newSync() without parameters) and not for use with node. Wrong use will just reliably deadlock.

```
<img src=favicon.ico>
```

```
</pre>
```

```
<script src="lib/pot-web.js" wasm="lib/pot.wasm"></script>
```

```
<script>
```

```
function onWasmLoaded() {
```

```
    map = pot.newSync()
```

```
    map.putSync("hello", "POT!")
```

```
    value = map.getSync("hello")
```

```
    console.log("key <hello> has:", value)
```

```
}
```

```
</script>
```



EXAMPLE 5: NETWORK

This page connects to a local Swarm network instead of using in-memory storage.

Except for the fact that it supplies the relevant network parameters to `pot.new()` it is in principle the same as example 1.

The source code below is for illustration, the batch id would have to be updated to run it. (It works when using `make example5` because `make` updates the source code with the latest batch id.)

- in-browser
- network persistence
- asynchronous

`examples/example5.html`

```
<pre>

POT JS Example 5: Network

Check source and browser console.


This page connects to a local Swarm network.


To test it, use `make example5`, which will start a local Swarm network
and update the batch id literal in the pot.new() call in this source
after creating a new batch or re-using one created before.


<img src=favicon.ico>

</pre>

<script src="lib/pot-web.js" wasm="lib/pot.wasm"></script>

<script>

(async () => {

  await pot.ready()

  map = await pot.new("http://localhost:1633",
    "eaec980d7a7b63d6a8f37f99cd815b318402e93bf2227a48c40d4d80d608915d")
```



```
await map.put("hello", "POT!")

value = await map.get("hello")

console.log("key <hello> has:", value)

ref = await map.save()

console.log("tree root is:", ref)

}())

</script>
```

WHAT YOU SHOULD SEE

BROWSER CONSOLE

[illegible]

EXAMPLE 6: NETWORK II

Example 6 works like example 5 but receives the batch id as a parameter.



- in-browser
- network persistence
- asynchronous

examples/example6.html

The source looks like `examples/example5.html`, except for the highlighted lines, which take the batch id from the browser URL.

```
...  
    await pot.ready()  
  
    batch = new URLSearchParams(window.location.search).get('batch')  
  
    map = await pot.new("http://localhost:1633", batch)  
  
    await map.put("hello", "POT!")  
...
```

The output is the same as with example 5.

EXAMPLE 7: NODE

This example shows how POT JS works with `node.js`.

The one difference vs. in-browser use is that `pot-node.js` is required instead of `pot-web.js`.

This example also shows how logging verbosity can be changed: setting `global.potjs_verbosity` to 0 suppresses all logging from POT JS, including even the first `pot: » POTWASM`. The preferred way of setting the verbosity level is shown in the next example, example 8: using the method `pot.setVerbosity()`.

- `node.js`
- in-memory
- asynchronous

examples/example7.js

```
console.log(`
```



POT JS Example 7: Node, in-memory, w/o pot-node.js

Check out the source in examples/example7.js.

```
`)  
  
fs = require("fs")  
require("./lib/wasm_exec")  
  
var go = new Go()  
  
global.potjs_verbosity = 3  
  
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)  
  .then((r) => { go.run(r.instance) })  
  
onWasmLoaded = async () => {  
  
  console.log("wasm loaded-function called")  
  map = await global.pot.new()  
  await map.put("hello", "node")  
  value = await map.get("hello")  
  console.log("hello:", value)  
}
```

WHAT YOU SHOULD SEE

TERMINAL:

```
G00S=js G0ARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go  
● ✓ pot.wasm built for production  
node examples/example7.js
```

POT JS Example 7: Node

Check out the source in examples/example7.js.

```
pot:  » POTWASM  
pot:  » node.js detected  
pot:  » init done  
wasm loaded-function called  
pot:  » initialize new P.O.T.  
pot:  » created new in-memory persister  
pot:  » using in-memory persister  
pot:  » ready
```



```
pot: » put hello: node
pot: » get hello: node
hello: node
```

EXAMPLES 8 AND 9: WEB APP

These examples demonstrate a single-source web app. The page is served from the server script. The POT functions run on the node.js server.

Examples 8 and 9 are identical to make a point: that the storage destination for the POT can be switched out by parameters. In this case, from in-memory storage (8) to local network (9).

example8.js and example9.js

```
console.log(`
    POT JS Example 8: Node.js Web App Server
    This is a self-contained web app, serving
    a page to receive input, returning results.
`)

const http = require('http')
const qs = require('querystring')
const form = `<pre>

    POT JS Example 8: Node.js Web App

    <form method=post action="http://localhost:3000">
    key    <input name=key>
    value  <input name=value>
           <input type=submit name=button value=put>
    <hr />
    key    <input name=search>
           <input type=submit name=button value=get>
    value  <input name=result>
    </form>

    </pre>

    <script> console.log('see server log') </script>`
```



```
const fs = require("fs")
require("../lib/wasm_exec") // note the ./

var go = new Go()
var kvs

WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
  .then((r) => { go.run(r.instance) })
global.onWasmLoaded = () => {
  bee = process.argv[2]
  batch = process.argv[3]
  console.log(bee, batch)

  kvs = pot.newSync(bee, batch)
  console.log('srv: KVS initialized')

  // Notes:
  // * The "pot: " log entries are coming from Go pot.wasm.
  // * There is no catching of early calls of put() and get()
  // * onWasmLoaded could be defined async, for await pot.new()
  // * onWasmLoaded has to be added to global to get detected
}

async function put(key, value) {
  await kvs.put(key, value)
  return `put ${key}: ${value}`
}

async function get(search) {
  value = await kvs.get(search)
  return `get ${search}: ${value}`
  <script>
    document.getElementsByName('result')[0].value = `${value}`
  </script>
}

const server = http.createServer(function(request, response) {
  if (request.method == 'POST') {
    var body = ''
    request.on('data', function(data) {
      body += data
    })
    request.on('end', async function() {
      var log
      const post = qs.parse(body)
      if(post.button == 'put')
        log = await put(post.key, post.value)
      else
        log = await get(post.search)
      console.log("srv: " + log.match(/.*/)[0])
      response.writeHead(200, {'Content-Type': 'text/html'})
      response.end(form + '<hr><br><pre>\t\t' + log)
    })
  }
})
```



```
    else {
      response.writeHead(200, {'Content-Type': 'text/html'})
      response.end(form)
    }
  })
  .listen(3000, '127.0.0.1')

console.log(`srv: listening at http://127.0.0.1:3000`)
```

WHAT YOU SHOULD SEE

BROWSER:

POT JS Example 8: Node.js Web App

key

value

key

value

put A: 123

The screen will refresh with fields empty on every button press.

TERMINAL:

```
% make example8
G00S=js G0ARCH=wasm go build -o lib/pot.wasm potjs.go nomock.go
● ✓ pot.wasm built for production
open http://127.0.0.1:3000/examples/example8.html
node examples/example8.js
```

POT JS Example 8: Node.js Web App Server



This is a self-contained web app, serving a page to receive input, returning results.

[illegible]



BUILDING

There is no need to build POT JS to use it.

The main executable, the Go WASM file `pot.wasm`, the Javascript and HTML files are portable.

To build, run

```
% GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go  
% cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
```

or use

```
% make build
```

WHAT YOU SHOULD SEE

```
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go  
✓ created pot.wasm for production
```



CLEAN BUILD

To rebuild from scratch, including integrity hashes and rebuilding `go.mod`, use:

```
% make clean build
```

WHAT YOU SHOULD SEE

```
rm -f go.mod
rm -f pot.wasm
rm -f wasm_exec.js
rm -f potjs.js.sha384 wasm_exec.js.sha384
rm -f .batch_creation .batch_id
go mod init potwasm
go: creating new go.mod: module potwasm
go: to add module requirements and sums:
    go mod tidy
go mod edit -replace github.com/ethersphere/proximity-order-trie=github.com/ethersphere/proximity-order-trie@v1.0.0
go get
go: added github.com/beorn7/perks v1.0.1
go: added github.com/cespare/xxhash/v2 v2.2.0
go: added github.com/ethersphere/bee/v2 v2.5.0
go: added github.com/ethersphere/proximity-order-trie v1.0.0
go: added github.com/hashicorp/errwrap v1.0.0
go: added github.com/hashicorp/go-multierror v1.1.1
go: added github.com/prometheus/client_golang v1.18.0
go: added github.com/prometheus/client_model v0.6.0
go: added github.com/prometheus/common v0.47.0
go: added github.com/prometheus/procfs v0.12.0
go: added golang.org/x/crypto v0.23.0
go: added golang.org/x/sys v0.20.0
go: added google.golang.org/protobuf v1.33.0
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" .
openssl dgst -sha384 -binary potjs.js | openssl base64 -A > potjs.js.sha384
openssl dgst -sha384 -binary wasm_exec.js | openssl base64 -A >
wasm_exec.js.sha384
for fn in potjs.js.sha384 wasm_exec.js.sha384; do sed -i.bak -e "s/\\(\\\"${fn:0-7}\\\" integrity=\\)[^>]*\\/\\1\\\"sha384-${$(cat ${fn})\\/\\/\\[\\/]\\/\\[\\/]\\/\\\"/"
example1.html ; rm -f example1.html.bak ; done
GOOS=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
✓ created pot.wasm for production
```



REQUIREMENTS

POT JS includes the Go implementation of POT.

POT JS API

Tested with:

- go 1.24.0
- node.js 22.17.1
- chrome 138.0.7204.184 arm64

probably works with earlier versions.

GO POT

- go 1.24.0
- testify 1.8.4
- swarm bee 2.5.0
- crypto 0.23.0
- sync 0.14.0

Which will require

- perks 1.0.1
- xxhash 2.2.0
- go-spew 1.1.1
- errwrap 1.0.0
- go-multierror 1.1.1
- text v0.2.0
- go-diffli 1.0.0
- prometheus
- x/sys 0.20.0
- protobuf 1.33.0
- yaml 3.0.1

See `go.mod`.



TESTS

To test the JS API, serve the main directory, e.g., with `npx http-server` .
and open `http://127.0.0.1:8080/test.html?tag=in-mem`.

Or use

```
% make test
```

It uses `test.js` and `test.css` to display the results from a suite of 142 tests across the available `put` and `get` functions, and more.

WHAT YOU SHOULD SEE

TERMINAL

```
G00S=js GOARCH=wasm go build -o pot.wasm potjs.go nomock.go
✓ created pot.wasm for production
○ start http server (stop with 'make stop')
(npx http-server -c1 . &)
open "http://127.0.0.1:8080/test.html?tag=in-mem"
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

[2025-08-30T21:44:02.883Z] "GET /test.html?tag=in-mem" "Mozilla/5.0 (Macintosh; Intel
Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
(node:22323) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is
deprecated
```



```
(Use `node --trace-deprecation ...` to show where the warning was created)
[2025-08-30T21:44:02.904Z] "GET /test.css" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.906Z] "GET /wasm_exec.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.910Z] "GET /favicon.ico" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.989Z] "GET /test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.990Z] "GET /kvs_test.js" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
[2025-08-30T21:44:02.997Z] "GET /pot.wasm" "Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36"
```

BROWSER



SWARM POT JS API TEST SUITE

IN-MEM

Sat Aug 30 2025 23:44:02 GMT+0200 (Central European Summer Time)

This is the test suite for the Javascript API to the Go implementation of the Proximity-Order-Trie (POT).

Also see: [example 1](#) [example 2](#) [example 3](#) [example 4](#) [read me](#)

NOTES

The test suite runs **in-memory** of the Go POT implementation.

Tests are using different random byte sequences for keys and values every run.

NETWORK

Bee node URL : (in-memory)

Batch ID : (none)

KVS TESTS

15 suites · 142 cases · 1325 assertions · passed with no errors.

Test Suite #1 ... EXAMPLE TEST ...

IN-MEM

Test setup test outside of main test files.

case #1 » html-inline

test.html:109

· put and get K: V

test.html:110

✓ as expected, ✓V.

5ms

Test Suite #2 ... SIMPLE GETS AND PUTS OF KVS WITH ONE ITEM, SYNCHRONOUS ...

IN-MEM

**BROWSER CONSOLE**

```
...  
wasm_exec.js:22 pot: + case #140 » Cancel Delay  
wasm_exec.js:22 pot: <object>  
wasm_exec.js:22 pot: create hanging test promise and cancel it (delay, chain .catch)  
wasm_exec.js:22 pot: sleeping  
wasm_exec.js:22 pot: canceled!  
wasm_exec.js:22 pot: ✓ expected error: 'Error: canceled'  
wasm_exec.js:22 pot:  
wasm_exec.js:22 pot: + case #141 » Immediate Cancel II  
wasm_exec.js:22 pot: <object>  
wasm_exec.js:22 pot: create hanging test promise and cancel it immediately (chain  
.catch)  
wasm_exec.js:22 pot: sleeping  
wasm_exec.js:22 pot: canceled!  
wasm_exec.js:22 pot: ✓ expected error: 'Error: canceled'  
wasm_exec.js:22 pot:  
wasm_exec.js:22 pot: *** Test Suite #14 *** FAILURES ***►IN-MEM◄  
wasm_exec.js:22 pot: Failure tests are available in extended API test mode. Run `make  
xt`.  
wasm_exec.js:22 pot:  
wasm_exec.js:22 pot: *** Test Suite #15 *** PROOFS ***►IN-MEM◄  
wasm_exec.js:22 pot:  
wasm_exec.js:22 pot: + case #142 » Return a Proof  
wasm_exec.js:22 pot: <object>  
wasm_exec.js:22 pot: • new map  
wasm_exec.js:22 pot: » initialize new P.O.T.  
wasm_exec.js:22 pot: slot ref: 101  
wasm_exec.js:22 pot: ✓ no error raised.  
wasm_exec.js:22 pot: • get proof for K1  
wasm_exec.js:22 pot: ✓ as expected, 'null'.  
wasm_exec.js:22 pot:  
wasm_exec.js:22 pot: *** tests cases run: 142 • assertions: 1325 ***  
wasm_exec.js:22 pot: ► no errors ◄
```

The standard tests run against the Go implementation of POT, like the use would be in production. The **extended tests**, however, focus on the JS AP to Go, add cancellation and time out cases. They do not interface with the Go POT code.

Latency emulation, promises, and connectivity as well as retention error conditions will be added to emulate the reality of an unreliable network as ultimate storage layer. Run the tests with `make test` and `make xt` respectively.

Note that `make xt` (specifically, `make mock` change the project configuration and build executables (especially a `pot.wasm`) that will not work for production. (They will appear to work but they sidestep Go POT.) Use `make unmock`, `make test` or `make build` to restore to the production target (i.e., build the real `pot.wasm`).

The **extended tests** use a separate, dedicated, in-memory mock storage to separate them from the Go POT implementation.



The tests are made for the browser as the final destination of the combined architecture, in either case (standard and extended tests) they are invoked by opening the file ``test.html``. The very small file that is altered between the test modes is ``testmode.js``, and ``go.mod`` to replace package the real Go POT with ``mock``.

This test page and test framework is designed to bridge the language divide and allow for the emulation of exceptions in one degree of separation (caused in Go, caught from JS). The major relevance of the test harness is to help surface any friction between JS's and Go's resource release mechanisms (JS promises and Go channels) to detect possible leaks. Go's channels are not without peril – they can deadlock – and the challenge is aggravated by their position behind the JS API. The relevant code triggering the exceptional conditions resides in ``mock.go``. For its purposes, it has to be part of the ``main`` package rather than package ``pot`` in ``mock``.

DEVELOPING

This chapter is about contributing to or changing POT JS itself.

Developing and testing at this point have been done on Mac only.



FILES

README.MD	subset of these instructions
potjs.go	Go JS interface
nomock.go	No-op version of test functions
mock.go	monkey patch mock of Go POT for extended testing
mock/	replacement of Go POT for extended testing
pot.go	
go.mod	
pkg/persister	copy of Go POT in-memory storage to satisfy dependencies
lib/	
pot.wasm	Go POT implementation compiled to WASM
wasm_exec.js	generic Go WASM wrapper (replace w/your Go version's)
potjs.js	Thin JS initialization convenience code
wasm_exec.js.sha384	SHA384 checksum for sub-component verification
potjs.js.sha384	SHA384 checksum for sub-component verification
doc/	
POT JS manual.odt	
POT JS function reference.pdf	
POT JS manual.pdf	
POT JS function reference.odt	
examples/	
example1.html	briefest example as listed above, in-memory storage
example2.html	interactive example, input fields, integrity
example3.html	example 2 dropping potjs.js for more control
example4.html	example 1 using synchronous access functions
example5.html	example 1 but with connection to a storage network
example6.html	example 5 with better connection to make rules
example7.js	example 1 for execution with node.js
example8.js	single-script node.js app, server and web page
example9.js	example 8 verbatim stared with different arguments
favicon.ico	Swarm logo to suppress browser errors
test/	
test.html	test main page for test, ext_test, locnet_test
kvs_test.js	test suites
test.js	generic test harness
test.css	generic test optics
go.mod	go module locations
go.sum	go module check sums
Makefile	build scripts
favicon.ico	Swarm logo to suppress browser errors



MAKE

The project comes with an extensive **Makefile** to support developing.

Below are the build rules for executables and for running tests of POT JS.

Note that building and any of the following is not required for using POT JS. The relevant executable files are portable and part of the repo. After cloning or downloading a distribution of POT JS, all relevant files are immediately usable.

RULES

Run with **make** `<rule>`

build	build executables
example<n>	start server and run example <code><n></code> . <code>n = 1-8</code>
test	explain test modes and run <code>node_inmem_test</code>
web_inmem_test	test api interaction with go pot in-memory persisting, web
web_inmem_stress	stress test with go pot in-memory persisting, web
web_sim_test	extended exceptions tests w/out go pot connection, web
web_locnet_test	standard tests with a locally installed Swarm network, web
web_locnet_stress	stress test with a locally installed Swarm network, web
node_inmem_test	test api interaction with go pot in-memory persisting, node
node_sim_test	extended exceptions tests w/out go pot connection, node
node_locnet_test	test with a locally installed Swarm network, node
clean	prepare for building from scratch
distclean	prepare for commit to repository, including build

SUPPORT & DEBUGGING

For day-to-day developing, these commands may be helpful.

serve	start local http server serving current directory
stop	stop local http server. Ctrl-c does not.
mock	prepare for extended exception tests
unmock	reset for production or non-extended tests
locnet_install	install the local test network
locnet_start	start the local test network on docker
locnet_batch	buy stamp batch (free)
locnet_tests	run test suites on local test network
locnet_stop	shut local test network down
locnet_logs	show the entire log of the queen node



DEVELOPER NOTES

A JS API IMPLEMENTED IN GO

POT JS, while being a Javascript API to Go code, is intentionally and necessarily programmed almost exclusively in Go. It would not work the other way around, i.e., by implementing more – or all – of its functionality in Javascript. But it is also desirable that it has no Javascript layer to speak of.

The exception, the small `potjs.js` script, is a dozen lines convenience code that replaces a few lines of initialization code that one otherwise had to write for oneself, if `potjs.js` was not used (see `examples/example2.html`). This is an option, `potjs.js` can be left out. In effect, the WASM executable, all written in Go, and nothing written in Javascript, constitutes all that is required to use POT JS.

There is no alternative to this design. But eventually this focus on Go is not a disadvantage. The following explains why it is necessary but also desirable to have POT JS programmed almost exclusively in Go and details how that is the cleaner solution.

◇

POT JS is really Javascript without Javascript code: even though POT JS is written in Go, what this Go code creates are Javascript objects and functions. For all intents and purposes, POT JS, at runtime, is mostly Javascript. The Go code creates Javascript runtime elements that constitute the actual API. That is, methods like `kvs.put()` are functions written in Go but executed by the Javascript runtime like native Javascript functions. The Go runtime's part is just to create them.

There is no way to do the opposite using `syscall/js`. I.e., there is no way to create native Go objects from Javascript. `syscall/js` is a Go package after all, it extends Go, not Javascript. There are translators that create Go code from Javascript code, at compile time. But that is not the same as creating runtime objects for one language in the other at runtime, which is needed to create an API as tight and fast as POT JS.

That `syscall/js` works in this direction, Go to Javascript, is plausible, because the calls are coming from Javascript to Go. We want a gate from Javascript to WASM Go code, not the other way around. This is why it makes sense that the connecting elements of the API are created in Go – using `syscall/js` functions – to create Javascript objects and Javascript functions. In a pedantic sense, while the code is all Go, the API actually is fully Javascript. Just created by Go code, not Javascript code. Eventually, Javascript does not 'realize' that it is operating on 'non-native' Javascript objects that are created using Go and `syscall/js`.

Apart from that, there is a reason why at least some Go code is required and the mirror image to write the entire API in Javascript would not be possible: there are two typical classes of Go objects in play, employed by Go POT, as any Go project would: contexts and channels. Because they cannot be transferred over to Javascript that does not know the concept of channels (contexts are special



channels), they have to be managed on the Go side. This is where syscall/js incompleteness might be hurting. Maybe there could be a way to hand handles to channels over to Javascript. This would ver likely require manual handling of garbage collection on the Go side though to prevent the channels from being discarded as unused because the Go runtime has no visibility on the Javascript side. The overhead of keeping track of contexts and channels – either by keeping the objects, or by protecting them from premature garbage collection – necessarily has to be implemented on the Go side.

Therefore, even in its slimmest form, using syscall/js, one would always end up having at least some Go code in the mix. The API could never be all Javascript the way that it is now (almost) all Go.

This also trumps another reservation that might be raised: that it should seem much cleaner if one could avoid adding a main package to the Go implementation of POT in order to create a Javascript API, and to instead let Go POT remain untouched, as is, with no additional Go code added just for this specific API. Convincing as this view is, it is not feasible because of the way syscall/js works. It is a given that API-specific Go code has to be added. It can't work to just wrap Go POT into WASM and be done, all the rest being in Javascript. There has to be some Go code that creates the bridge; it can't be Javascript code.

And that Go code has to be part of the WASM. Thus only compiling Go POT to WASM won't work. One way around it would be to make the WASM act like a server, which would be slow, not the implementation of a Javascript API, but a general interface. And it would require adding code to Go POT just the same. But to be exact, Go POT is not changed, it is just made the main component of the API project POT JS. What is more unsatisfying is merely the question of how to organize the repos.

So, much of the objects and functions the Go code creates are Javascript. Thus, much of POT JS is really Javascript, just not Javascript code. Something must be 'the bridge' and these Go-created Javascript objects are it.

◇

Now, if the question was asked from a high level, whether having virtually NO Javascript code can be a good choice for a Javascript API, the answer is that the nature of syscall/js to mainly offer the possibility to 'program Javascript in Go', but not vice-versa, determines all else. However, even those aspects that would appear to unquestioningly lend themselves to be implemented in a complementary Javascript layer, rather than in the Go code, turn out to only work when programmed in Go, instead of in Javascript.

This is a major challenge that adds to the picture: transacting between Javascript and Go can deadlock the Go code when certain functionality is written in Javascript, instead of in Go. This even includes the creation of something as close to home as Javascript promises.

As an epitome of Javascript programming, it must seem strange that POT JS creates them in Go. However, it simply doesn't work in the way that would look more intuitive, i.e., creating a Javascript layer that defines the promises, which's executors then call into the Go WASM functionality synchronously. That this does not work, as much as it would seem that it should, can easily be confirmed by trying it out. Below is some background that may help to explain why this is so.



In the end, the answer to why POT JS is mostly programmed in Go is, it is not a choice.

But beyond these hard facts that favour coding POT JS in Go, there are reasons, given below, why it can be regarded as optimal design. The right choice, if it was not inescapable.

◇

From a high-level it is preferable to have code only, or mostly, on one side – Javascript or Go – and to not cross the language barrier multiple times, to keep things robust and avoid unknown unknowns. Keeping things mostly on one side can minimize error sources and overhead that may occur unwittingly when switching between languages, as well as the loss of original state that this entails (channels on the Go side, exceptions for Javascript). This is why it looks preferable, independently of the limitations that partially dictate the answer, to keep the bulk of code either on the Go or the Javascript side.

The way syscall/js works, this predisposes the Go side then.

Keeping things simple – by avoiding language switch overs – becomes more relevant for the disclaimer that comes with syscall/js, that it is “experimental and not for production.” There seem to be no complains that it is not stable. It may be Google’s typical traditional caution. But it encourages to keep things simple and straight forward and to avoid demanding constructs where possible. Keeping it all on the Go side then, after some code on the Go side is unavoidable in the first place, appears to be the right choice to increase robustness.

But even if all that wasn’t acceptable as convincing argument for pushing everything into Go, it turns out there is a technical killer argument: as mentioned, it does not work to have asynchronicity being taken care of on the Javascript side. It does not work to have the promise wrapping for asynchronous calls be on the Javascript side. And when using WASM with node, it’s not only the fetch call that is locking up – as the syscall/js documentation warns – but every synchronous call deadlocks (sic), including when using in-memory storage as persister, which is not using a fetch internally. It does not work to create the promises in Javascript for the purpose of a Javascript-side layer. They have to be implemented in Go, as POT JS is doing. Specifically, what does not work is to wrap sync calls to Go WASM in Javascript promise executors on the Javascript side.

But using ‘the same’ logic creating the Javascript promises on the Go side, works:

<https://github.com/brainiac-five/potJavascript/blob/86fc8af3ea6b0f148bed1efb34d8dd00190bd1b3/potJavascript.go#L50>

The syscall/js documentation is not clear that this would be required to avoid deadlocks. There is a description on how deadlocks should be avoided by using go routines. But it is not explained why creating the asynchronicity on the Javascript side should be less effective. Under certain conditions, this deadlock happens every time. What causes it seems to not only be the browser’s main fetch thread, because it also affects node.js, and even more profoundly. With node.js, The deadlocks happen reliably unless the Javascript promises are created on the Go side. Which in fact entails a go routine to uncouple the promise executor from the synchronous program flow. Just as the syscall/js documentation seems to propose. While the Javascript asynchronicity inherent in the promise creation, empirically, does not suffice.



This might be because in a 'go <executor>' call the go routine call's mechanics are truly multithread, implemented such that if the operating system allows, go routines might run on parallel threads. Maybe this is what is preventing the whole mechanism from locking up by virtue of a stronger decoupling effected by a go routine call. In that interpretation, the fact that doing the same on the Javascript code side – wrapping sync API calls into Javascript promises – always locks up could be explained by the fact that in Javascript nothing is ever truly parallel, as a basic strategy to keep Javascript programmers safe even while the paradigm is strongly async (except Web Workers, which add exactly that: multithreading). Go routines though are designed to possibly run in parallel if the OS and hardware allows for it. Conversely, this thesis is weakened by the circumstance that Go in WASM is said to always run on only one process.

Regardless, calling from Javascript into Go and then Go calling back to Javascript implicitly by making a fetch call seems to only ever work by go routine decoupling. When using node.js, there seems to be an additional cause for reliable deadlock in play when not decoupling calls with a go routine. Always building such a go routine into a Javascript promise makes for a very desirable architecture though that has no lamentable drawbacks. It even has the precious advantage that such Go-native promises provide the only way to trigger native Javascript exceptions from Go.

As the promises are the 'most outside' hull of calls though and seeing that even they cannot be implemented on the Javascript side, little else can.

But as noted, because for the sake of robustness it looks desirable to have as much code as possible on one side, and to avoid crossing language lines within one call, the necessary design also looks desirable. Even if it is surprising that eventually there is no real Javascript layer.

This is why POT JS is implemented almost entirely in Go, and why this is a desirable outcome, too.

NOTES ON DEBUGGING

Being wrapped in WASM to interface with Javascript, the Go executable is opaque. It cannot be debugged as there are no debuggers for WASM, and it becomes unavailable once it has panicked.

Consequently, developing benefits from being test-driven to unearth unexpected problems.

A wide array of tests are available in test/kvs_test.js. They include examples of failure test of varying nature.

Logging directly to terminal or browser console can be done with the Go function log(). It uses the interface itself to have direct access to Javascript's console.log.



NOTE ON SYSCALL/JS

The connection between Go and Javascript through WASM is based on Go's `syscall/js` package. This package is officially labeled as experimental by its provider, Google:

"This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise."

<https://pkg.go.dev/syscall>

This is a stark warning going beyond Google's typical caution. It may explain the incompleteness of `syscall/js`, e.g., the lack of an option to directly throw exceptions. What remains unclear is why this imperfect state has been going on for so long.

This also makes it look possible that, e.g., the `node.js` WASM crashes that report deadlocks are not in fact always deadlocks but sometimes a misfiring, overzealous deadlock detection. The Go WASM executable stops at any rate. The insight would not be the path to a solution.

INITIALIZATION

Starting out, the Go WASM executable has to be loaded, both for a HTML5 application or when using POT JS with node. In the browser this can be handle conveniently by including `potjs.js`, which also provides the wait function `pot.ready()` The core of `potjs.js` are only a few lines of code and they can alternatively be programmed directly in your source code, omitting `potjs.js`, as shown in `example2.html`. In some situations that might give helpfully more fine-grained access to design the start-up sequence (the example happens to use `newSync()`):

```
WebAssembly.instantiateStreaming(fetch("pot.wasm"), go.importObject)
    .then((r)=>{
        go.run(r.instance)
        map = pot.newSync()
        ...
    })
```

Generally, only after the `pot.wasm` executable has loaded – and in the course, started and is running – is the global `pot` object created and the POT JS functions described above become operational. Before that point, the `pot` object and the functions do not exist and trying to use them will cause the according JS exceptions to be thrown.



But if `potjs.js` is used, it creates the global `pot` object earlier, before `pot.wasm` is done loading and starting up, and provides through `pot.ready()` a promise that can be used to wait for `pot.wasm` to complete loading. But `pot` and `pot.ready()` are the only elements of the API that already exist at that point, no other function does until the promise of `pot.ready()` resolves.

```
await pot.ready()
map = await pot.new()
...
```

Without `potjs.js`, `onWasmLoaded()` can be created instead, which will be called by `pot.wasm` as soon as it is ready:

```
function async onWasmLoaded() {
  I      map = await pot.new()
  ...
}
```

The `pot.wasm` executable loads quickly. Most of the time, it will not be strictly needed to use `pot.ready()` or `onWasmLoaded()`, as demonstrated by `example2.html`. The interaction coming from the user will practically always happen only after `pot.wasm` has long been loaded.

This can be a source of errors though in the case of a network hic-up, and interoperation in a node.js program will generally be too fast to ignore the need to wait for `pot.wasm` to finish loading.

TYPES

A Javascript developer will make assumptions about how types work. To provide the least surprising solution,⁷ Javascript types are automatically converted to and back from the raw binary storage format values are stored in to come back as-stored.

The Go POT implementation stores raw binary data. In Javascript the equivalent would be `Uint8Array`. But that is not idiomatic to normal Javascript use. `Uint8Arrays` cannot be used for keys,⁸ and cannot be compared with each other as would be expected. They are generally less practical as values for a lot of use cases in Javascript, where the expectation would be that sending

⁷ en.wikipedia.org/wiki/Principle_of_least_astonishment – “In user interface design and software design, [1] the **principle of least astonishment** (POLA), also known as principle of least surprise, [a] proposes that a component of a system should behave in a way that most users will expect it to behave, and therefore not astonish or surprise users.”

⁸ Because they do not have a natural order in Javascript.



a variable of a certain type to a key-value store should later get you that variable in that type back, rather than some raw data that has to be type cast to be what it once was.

Javascript being weak-typed does not help, it rather heightens expectations regarding automation of type handling and on-the-fly casting.

To achieve type-awareness, type encoding has been implemented that can put and get Javascript values without losing their types. It writes a first byte to any **put** (that does not have **Raw** in its name) that reflects the JS type. This makes data stored with **put()** mostly incompatible with any get function that has **Raw** in its name.

But this is not the only way for a user to deal with types. There are *raw* access functions that deal with types on *getting*: **getBoolean()**, **getNumber()**, **getString()**. They are for reading data stored with **putRaw()**, where only one byte is stored for Booleans, 8 for numbers, etc. without the leading type byte.

This dual API for type-aware storing is separate to a third one that just stores raw **Uint8Arrays**, using **putRaw()**, and then **getRaw()**. This leaves the task to the user to cast to the right type.

NUMBERS

Javascript numbers are stored to Swarm as IEEE 754 floating point standard byte code.

The reason is that the universal number type in Javascript, *even integers* (other than the special case of **BigNums**), are internally stored as 64-bit floats. Direct access to the bits representing these numbers guarantees lossless conversion, especially in edge cases.⁹

The principally desirable representation of *integers* by their hexadecimal byte corollary was renounced as it would have meant special casing the number conversion, i.e., handling integers and floats differently depending on their face value not by any distinction Javascript itself knows or cares about. It would make some Javascript numbers into something they are not (integers), for little tangible advantage other than the hypothetical portability of POT storage between clients of different languages.

If cross-system portability of POT data access becomes a goal, then the higher robustness of an integer representation for integers would be convincing – *if* the input could reliably be known to have been meant as an integer in the first place, which in Javascript could be difficult to ascertain. Instead, an additional type marker could be added that identifies integers and allows Javascript to read them but not write them.

⁹ The alternative of storing numbers as strings works well, too, but goes through a much more involved process of conversion that can be assumed to be much more time and memory-consuming. While in the light of this context this does not matter too much, it might also be more prone to errors in edge cases such as the likes of **10/3**, **-0**, and *multiple* variations of **NaN** in IEEE 754.



BIG NUMBERS

Javascript Big Numbers could not be implemented at this point because the Go `syscall/js` package – being officially incomplete, see above – can currently not handle them. It seems likely that `syscall/js` could be extended to make **BigNums** work if desired. There are receipts for it online.

While modding the standard Go package for WASM communication with JS opens a can of worms, it seems important enough eventually to allow for **BigNums** to be stored in Swarm natively.

TESTS

To accommodate for the dual origination of errors – on the JS or Go side – the combined test suite utilizes the same JS ↔ WASM ↔ Go channel that the API itself uses to trigger, evaluate and log the tests, jostling between languages. A number of functions were added to enable black box testing. They are no ops in production, neutered via Go build directives.

To be able to test its reaction to delays, stalls, exceptions and cancellations, and to insulate tests of the JS API from any potential problems with the Go POT implementation itself, which is not yet battle-hardened or extensively tested, the ‘extended’ tests implement their own mock back end (**mock.go**). This is a second in-memory storage variant, separate from the in-memory storage the Go POT implementation itself offers per default and that is used in the ‘standard’ tests.

This mock back end, besides being helpful for debugging, exists to emulate fringe conditions that could not otherwise be tested but must be expected when the Go POT package operates connected to the Swarm network. E.g., mock delays are essential to test the basic user cancellation and time out design that has to satisfy both Javascript and Go language idiosyncrasies to prevent resource leaks in Go from dangling sub routines that were not freed when a request did not succeed. As a special complication, both runtimes have their own garbage collector, which always add opaqueness but, in the instance, have to be made to work together in their estimation of what resource can be discarded. In this regard, Go channels are a special challenge, as well as a potential source of deadlocks.

ERROR HANDLING

The divergent error handling philosophies of Javascript and Go had to be balanced. A Javascript programmer does not expect errors coming back as return values – like in Go – but as exceptions. But Go cannot trigger Javascript exceptions through WASM. It can **return** a value of type **Error** as JS-native **Error** type return value but not **throw** it directly, due to the incompleteness it seems of `syscall/js`.



While **throw** is not available to Go, a middle ground is provided by JS promises that enable – when created in Go instead of in JS – by the use of their **reject** function a means to trigger an exception in Javascript. This constitutes an additional reason for why the API functions that return promises are preferable. It allows for the most seamless transportation of errors from inside Go to Javascript in the expected way for a JS programmer. And calls from JS to WASM, one can read, should generally be promises as the nature of WASM is not quite like a library without agency but more like a parallel process. (It is not as it is executed on the same thread as the JS main thread).

Synchronous calls cannot benefit from this way of throwing errors. They are made to return a Javascript Error object instead of the expected return value. This does not fit well in the Javascript workflow and there are possible solutions. A promise can be created on the Go side in case of an error that immediately rejects. However, this still leads to asynchronicity, literally, between the error and the arrival of the error message, and “uncaught (in promise)” type errors that arrive a little bit late. E.g., often after a test program has finished.

A second solution is described in

<https://stackoverflow.com/questions/67437284/how-to-throw-js-error-from-go-web-assembly>: the WASM compiler can be extended using a hint, and the generic **wasm_exec.js** enhanced to add the possibility to **throw** from Go.

Because this seems only interesting for synchronous functions, which are not central to the overall user experience, the effort and the risk of touching the compiler and **wasm_exec.js** – where until now they are used without modification – seem not appropriate. The less than optimal solution will remain that synchronous functions return error objects when things go wrong, and that async functions that return promises should be preferred.

SLOTS

The array of **Slot** structures, **slots**, keeps the state of a KVS on the Go side that cannot be transferred to Javascript. On first glance, one would prefer a different, more elegant solution. But on inspection, they might be the best solution. Eventually, in what is a bit of a mine field, they are sufficiently simple that they can be made robust.

The slots hold the Go objects and channels for which how to create a safe JS handle for is not obvious and would create garbage collection issues. A handle for a Go object that could, e.g., be created from its memory address and given to JS, might have the effect that Go destroys the object if it detects that it has become unreachable by Go’s count. Go’s **runtime.KeepAlive()** is designed to work within one function. Maybe a strategically created closure could keep variables alive within it and could prevent the collection. But that starts to look error prone and no better than a straight array of state.

Another thinkable alternative to the **slots** array could be to use closures to hold the state itself even where it cannot be transferred to Javascript and attach those closures to the Javascript object that is



created for a new KVS. This works for the Go context `cancel()` functions of the JS promises. But while a Go function can be wrapped in a `js.Func` – a Javascript function that is created in Go and visible on both sides – and this closure could then be called by calling the wrapping JS functions from within Go (instead of from JS as they are intended), state cannot be accessed from Go when wrapped in this way because you don't get it back out. The return value of a `js.Func` must be a JS object. This cannot return anything that is not convertible to JS, like channels are not. It is visible in the `syscall/js` package's source code that the implicit, automatic conversion from Go to JS values cannot help to cross this threshold as it has the same limitations. `syscall/js` could probably be modified to circumvent this block but that would have to include the task to make sure that the garbage collector stays up to date and starts to look too involved.

A yet more highwire solution could be found by enclosing Go variables in the `js.Func` closure that can then be accessed through a wrapping Go function because the variable exists in both the inner and outer closure. But the outer closure itself cannot be attached to the JS object and would still have to be held somehow on the Go side – e.g., in an array. One would again resort to a map or an array to hold the access at hand to the relevant original Go objects. There seems to be no way to transfer state alien to JS to JS and get it back out to Go. That's by design.

This is why a state array like `slots` is useful, even while the `cancel()` functions of the promises work without such a central list. The Go state is simply kept on the Go side.

The integer index of a `Slot` in the `slots` array + 1 is used as the handle that is given to the Javascript KVS object as a Javascript number. `this.slot_ref` on the JS side holds it. A number can be passed to Javascript without a problem – even if there is the theoretical caveat that by necessity the integer is converted to a float, as JS does not have native integers, except for `BigInts` that `syscall/js` cannot handle. Thus, theoretically a conversion to a string would be safer. But in practice, JS using 64 bit IEEE 754 floats, the first integer conversion to fail is 9,007,199,254,740,993.

The `Slot` struct is retrieved on every call to `get()`, `put()` etc. accessed through the `this` pointer that is the first parameter that every `js.Func`-wrapped Go function receives when it is called from JS. The slot index + 1 is the number that is stored in the `slot_ref` field of the Javascript KVS object. This is what is accessed through the `this` reference in a `put()` or `get()` call. `this.slot_ref` is used as index + 1 to access the right Go `Slot` struct of the KVS that is being operated on from the slots array. The +1 keeps 0 an invalid `slot_ref` as a safety measure.

The most important element in the `Slot` struct is the Go `SwarmKvs` pointer to the struct that is returned from `newSwamKvs()`. It holds, via Go `pot.SwarmKvs`, `pot.Index` that is the struct holding the channels that are required by `muxProcess()`, the function that loops through a `select` that sequences write access to a POT. Every KVS has its own. It kicks in when it is created with `pot.new()`.

ALTERNATIVES

It can seem that a different locking mechanism than `muxProcess()`'s `select` could be implemented on the Go side as Javascript's single-threaded nature could be taken to represent a free replacement for a multiplexer. Hypothetically, `muxProcess()` was not needed in its current form when interfacing



to Go POT from Javascript, neither from the browser nor from node.js unless Worker threads were employed; by the logic that Javascript is already single-threaded and needs no protection against parallel writing. One might even ask whether the doubling locking aspect of this doubling of sequencing (that JS because it is single-threaded by nature does nothing in parallel ever, which `muxProcess()` doubly makes sure does not happen for writes) might be what unintentionally causes the not quite explained deadlocks of *all* sync calls coming from node.js. Perhaps the channels and `muxProcess()`'s constantly waiting `select` might not be required, the channels could possibly be left out altogether or created per-call instead so that they do not have to be kept available between calls – in the `slots` array – with the KVS object. This could be a required modification to make JS and Go play together more nicely.

If this would work, it would leave only the second type of channels to deal with, the `contexts`. They could likewise be created per-call so that they would not have to be kept around between calls, at the cost of losing the possibility of a KVS-wide cancellation option.

The last relevant object held in Slots then would be the persister. As this is not currently keeping a connection alive it could in its production variant – that connects to a Bee node via http – be implemented in a form where its state could be transferred to Javascript, if it needs state at all. Alternatively, the persister could be architected to be a native Javascript object, implemented in JS (code) in the first place. The bigger obstacle in this case would be the special, state-holding, in-memory version of the persister that has to be kept alive between incoming calls from JS to not lose its state. But the overall architecture should not be bent to make this less-important, in-memory variant of the persister easy to implement. To the degree it is practical for testing, the persister code itself should be the place where the kludge should be residing that makes state-holding between calls possible; rather than burdening the overall architecture.

But even for such streamlined version of Go POT and POT JS where Go POT is altered to work better with POT JS, `muxProcess()` might turn out to not be superfluous. It is already daring to rely on Javascript to never really do things in parallel. That means banking on that there will never be two write calls overlapping each other coming from JS in a way that would trip up Go POT. Apart from the fact that some future browser optimization might break guarantees that are currently implicit but not spelled out. A browser might do something 'helpful' that ends up using more than one thread to speed JS up and by that break the single-thread assumption. This is not unrealistic because any measures such improvement would require building into the JS runtime would be invisible and unavailable when control is given over to WASM.

But even as is, JS' asynchronicity could be enough to have two write promise's executors end up switching back and forth in a way that they look parallel to the Go side. In the end the promises are there to make things look like they happen in parallel and have multiple things pending. That's exactly what `muxProcess()` is there to prevent for writes. Multiple writes can pend but they must not happen at the same time, and this sequencing is what `muxProcess()` provides. JS would push things up to the point where it looks like the writes are both happening and then wait for the result, it does not nicely queue them.

Finally though, Go compiled to WASM does itself only run on one thread and cannot use its potential to multithread. It is not the case that a speed up through parallelizing and multiplexing could be expected, with parallel writes for different KVS, even while parallel writes for a single KVS is what `muxProcess()` exists to prevent. It is also not in this constellation the case that it is a useful capacity



of Go that JS can't deal with and thus it has some high-level consistency that it has to park the controlling state of this Go superpower outside. Go itself can when compiled to WASM not do what **muxProcess()** is designed to prevent it from doing in the wrong moment: work things in parallel. It would be plausible to expect that **muxProcess()** at least prevents that two writes race each other by switching back and forth between the two in an unlucky way. However, it would be enough to not use go routines to make sure that no two writes could race each other because WASM is executed on a single thread by Javascript. The expectation has to be that **muxProcess()** is unnecessary when Go POT is used in WASM. Unlike its use in a different environment, e.g., when used in Go as a library or package of a bigger system, e.g., included in a Bee client, there is no need to take care of the case that **put()** calls could be coming in even on different green threads, much less truly parallelly.

Together with surprises and limitations that surfaced while joining Javascript and Go, it looks difficult to judge whether an attempt at improving this would be worth the try.

SLOTS MIRROR A POWER IMBALANCE

In the end, in a holistic view, the core of the problem is in the first place that the sequencer state – the **muxProcess()** channels – would ideally be carried over from Go to Javascript to be held there, which we can't and thus have the **slots** array for; while what's technically only needed, instead, would be to either make sure there is no asynchronicity started within Go POT during the handling of one **put()**, or to have a simple semaphore per KVS on the JS side that makes sure that no two write calls are triggered from JS at the same time. In the latter case, there could still be multiple promises for puts created and pending at about the same time, but they would simply have to be prevented from running their executors to the blocking point of a write at the same time, and not rely on asynchronicity alone that would avoid blocking for JS but not a write race on the Go side. Abstractly, this would seem to not change much, because at present, that queuing is happening in **muxProcess()** on the Go side. But it would be a simpler model, more robust and possibly apt to avoid the deadlock crashes¹⁰ of the Go WASM executable that might be false positive of an overly sharp deadlock detector of the Go runtime (that was historically too lenient).

But overall, making Go POT cater to JS's needs would be an optimization the time of which probably has not come. In light of all of the above, the simplistic slots array looks like the best solution. Its inelegant ways are somewhat explained by how it is an artefact of the overall architecture not being optimal from the JS perspective. With its possibility for true parallel execution, Go POT is more powerful than what JS can make good use of having it in WASM and figuratively speaking, that power needs to be parked somewhere, unused, in the form of the channels.

Go POT on the other hand is meant to have the capability to work truly in parallel, because on the one hand the index updates could with big tries tilt from network-bound into processing-bound. I.e., the wait-time could be caused by the calculations happening in Go POT proper rather than at all times be the wait for an answer back from the Bee API. On the other hand, how it is crafted is just the go way of implementing asynchronous calls to the storage network.

In this view, the construct of the **slots** array is an artifact that betrays a friction across the overall implementation, joining Go and JS, and stemming necessarily from that Go POT was not made as a

¹⁰ To be clear: that happen predictable and reliably, not



JS backend and can do more than JS can ask of it, namely work in a truly parallel fashion. The **slots** array deals in exactly those Go elements that JS cannot hold, i.e., the channels, for something it was not designed to do, real multi-threading. And as a consistent consequence it does not have the means to hold the special type of state either that is native to Go to make true multithreading possible.

NODE.JS SWARM PERSISTER

The topic of network access for WASM executables run by a node.js script is relatively sparsely discussed on the internet and some information found there is misleading. There is a simple answer:

NO NETWORK ACCESS FOR WASM UNDER NODE.JS

Different from a browser, Node.js does not allow any network access for a WASM executable. DNS, TCP, UDP, any fetch access fails from Go running in WASM when executed by node.js. It works in the browser though. This is partially documented but the situation is less than fully clear. In the end, WASM was entirely made for the browser – to uniquely allow for speed like compiled languages. That WASM could also be used with node.js was, as it seems, an afterthought. WASM with node.js is implemented to a degree as ‘emulation.’

At any rate, while in the browser, network access is extended to the WASM executable, the sandbox that restricts the WASM code when running in node.js is stricter. Plausibly, because there is no corollary with node.js to the sandbox that contains the *browser* itself and separates it from the filesystem. Node.js lives on the other side of that divide, with full local access.

HYBRID PERSISTER

A new **persister**¹¹ in the mold of the Go Swarm **persister** had to be written for node.js to work with the Swarm network. While the original Go Swarm persister works for POT JS when operating in the browser – i.e., Javascript running in `<script>` tags on a web page – it does not work when used with node.js, i.e., for Javascript projects run locally from the command line.

The new persister, **SwarmNodeJsLoadSaver**, is written in Go and largely identical to the Go **SwarmLoadSaver**. It delegates network access to a fetch function **jsFetchSync()** in Javascript that uses the XMLHttpRequest package **XMLHttpRequest-ssl**, with a modification, as described below, to realize synchronous network access as required for the persister architecture.

This is the essential step towards a pure Javascript persister, even while other details of it are not entirely clear yet. However, the bolted-on and hacky way that **XMLHttpRequest-ssl** is

¹¹ The persister is a separated-out module that realizes the most basic primitives to store and retrieve for Go POT. It can fundamentally only write data to whatever the target storage is or retrieve it from there. Go POT originally has two persisters, one that stores everything in memory and one that stores everything to a Swarm network.



implemented, together with the fact that there seem to be no better alternatives – it's a religious theme apparently to keep JS code async – casts a shadow over the idea of pure Javascript persisters. A browser-only version might be a strong decision. A node.js version, with the idea to have persisters be supplied by users (developers), would always have to find its own creative ways to realize blocking writes and reads, or use the modified version of `XMLHttpRequest-ssl`.

SYNCHRONOUS ARCHITECTURE

The Go POT implementation is fundamentally a synchronous architecture. It waits for writes to succeed before continuing its tasks, and has a locking mechanism that allows for only one write at a time. It is unclear whether the algorithms of POT itself demand blocking waits for every write, especially because the network writes are from cache and the immutable, content-addressed entries should be persistable in any order without causing race conditions. In practice, it makes sense to assume that this cannot easily be changed and will not be changed as it might be asking for major re-architecting of how Go POT works and how it is compartmentalized into modules.

Node.js, however quite successfully strives to make synchronous writes all but impossible. Its `fetch` does not have a synchronous option but works only as `promise`. `XMLHttpRequest`, which still works in the browser and can be used synchronously, has been retired for node.js. There exist npm packages to bring it back and they are so popular that one of them registers 6M downloads a week. This package employs a hack to evade Javascript's enforced asynchronicity: it spawns a child process that handles the network request asynchronously, then blocks, waiting for the child process to terminate, which it does after the response callbacks are completed. That this is the way, demonstrates how averse node.js is to synchronous network access, and it seems to be the only viable option to create synchronous network requests for node.js. The downside is the hacky way in which the child process is supplied with the necessary data, using code generated as a text string at runtime and `eval()`, and that it is incomplete. Headers are singled out as malfunctioning in the docs.

Worse, it cannot handle binary POST data but expects Javascript objects to stringify to JSON, i.e., HTML form-like payload. This is why the package was slightly extended and included in the POT JS repository in its modified form. It has an MIT license.

Requiring it as a dependency and then modifying it in-situ seemed less sensible as it could be installed in different locations and with limited access. The recent attacks on NPM are an additional factor to not want npm installs in the tool chain, if taken together with the fact that the package source has to be modified at any rate.



ROADMAP

Possible future improvements:

- **FUNCTION TIMEOUTS**
- **PROOF RETRIEVAL FUNCTION**
- **TREE WALK FUNCTIONS**
- **GO-SIDE RESOURCE TRACKING INSTRUMENTATION**
- **JS-NATIVE PERSISTERS**
- **USE OF JS NEW OPERATOR**
- **INTEGRATION TESTS**